풀이 및 채점기준

2020년 1학기 C프로그래밍 2반 (40121-2) 기말시험

(2020년 6월 26일 금요일 오전 10시~11시 50분; 총5쪽 5문제, 총점 100점)

(오픈북/온라인 시험)

백지에 "C프로그래밍 2020년 1학기 2반 기말시험"/학번/성명 기재 후, 풀이와 답을 기재

시험 종료 시, 답지를 촬영하거나 스캔하여 jpg 또는 pdf 파일로 만든 후, 하나의 zip 파일에 담아

파일명을 C2F(이름)(학번 마지막 자리 수).zip(예: C2F홍길동9.zip)으로 하고, 이를

반드시 학교 이메일 계정을 사용하여 아래 강사 이메일 주소로 송부

김용한 교수(yhkim@uos.ac.kr)

아래 모든 소스 프로그램은 인텔 CPU를 탑재한 컴퓨터의 윈도우10 OS 상에서 Visual Studio 2017 Debug 및 x86(또는 Win32) 모드를 사용하여 빌드하고 실행하였다.

문제 1. (20점) 아래 소스 프로그램과 그 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라. 단, 프로그램 자원(resource)를 최소화할 수 있는 형태로 답을 제시하여야 한다. C 언어 문장 마지막에 ';'을 누락하지 않도록 주의할 것

```
#include <stdio.h>
#include <string.h>

void main(void)
{
    [redacted]

    while (1)
    {
        printf("주민번호 앞 6자리 입력: ");
        fgets(id[0], sizeof(id[0]), stdin);
        [redacted]

        printf("주민번호 뒤 7자리 입력: ");
        fgets(id[1], sizeof(id[1]), stdin);

        printf("주민 번호: %s-%s\n", id[0], id[1]);
    }
    printf("END\n");
}
```

문제 1-1 (C 언어 1문장임)

문제 1-2 (C 언어 1문장임)

```
Microsoft Visual Studio 디버그 콘솔
주민번호 앞 6자리 입력:
주민번호 뒤 7자리 입력:
주민 번호: 123456
-1234567
주민번호 앞 6자리 입력: 주민번호 뒤 7자리 입력: E
주민 번호: 8
-E

주민번호 앞 6자리 입력: E
END

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로그래밍₩6-기말시험
인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 1-3

문제 1-4

문제 1 끝.

답:

```c
#include <stdio.h>
#include <string.h>

void main(void)
{
    char id[2][8];

    while (1)
    {
        printf("주민번호 앞 6자리 입력: ");
        fgets(id[0], sizeof(id[0]), stdin);
        if (id[0][0] == 'E') break;

        printf("주민번호 뒤 7자리 입력: ");
        fgets(id[1], sizeof(id[1]), stdin);

        printf("주민 번호: %s-%s\n", id[0], id[1]);
    }
    printf("END\n");
}
```

```
Microsoft Visual Studio 디버그 콘솔
주민번호 앞 6자리 입력: 123456
주민번호 뒤 7자리 입력: 12345678
주민 번호: 123456
-1234567
주민번호 앞 6자리 입력: 주민번호 뒤 7자리 입력: E
주민 번호: 8
-E

주민번호 앞 6자리 입력: E
END

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로그래밍₩6-기말시험
인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 1-1의 답: `char id[2][8];`

문제 1-2의 답: `if (id[0][0] == 'E') break;`

문제 1-3의 답: 123456

문제 1-4의 답: 12345678

채점 기준:

각 소문항 당 5점 배점. 문제 1-1의 답에서 **char** 대신 **unsigned char** 또는 **signed char** 라고 답한 경우도 맞는 답임. 또 문제 1-2의 답에서 **id[0][0]** 대신 ****id** 또는 **(*id)[0]**라고 답한 경우도 맞는 답임. 부분 점수 없음

풀이:

(1) 일단 프로그램과 시행 예에서 가려진 내용을 모두 알고 있는 상태에서 프로그램의 동작을 설명한 후, 아래 (2)에서 답을 찾아가는 논리에 대해 설명하고자 한다.

fgets 함수는 입력 버퍼로부터 두 번째 인자가 지정하는 최대 바이트 수보다 하나 작은 바이트 수까지를 입력 버퍼로부터 읽어 와서 마지막에 널 문자에 해당하는 0값의 한 바이트를 첨부하여 완성된 문자열(즉 스트링)을 읽어오는 함수이다. 만약 지정된 최대 바이트 수보다 하나 작은 수까지의 바이트들을 읽기 전에 입력 버퍼가 비게 되면 현재까지 읽어 들인 바이트 데이터 다음 자리에 널 문

자를 저장하고 리턴한다. 만약 fgets 함수가 호출되었을 때, 입력 버퍼가 비어 있으면, 입력 버퍼로 바이트 데이터가 들어올 때까지 기다린다.

프로그램 사용자가 입력하는 내용은 라인 마지막에 <엔터 키>를 치면 그 때까지 입력한 글자들과 마지막에 친 <엔터 키>에 해당하는 아스키 코드 값에 해당하는 바이트 데이터들이 입력 버퍼로 저장된다. <엔터 키>를 치지 않는 한, 그 전에 입력한 글자는 입력 버퍼로 전달되지 않는다.

첫 라인에서 "주민번호 앞 6자리 입력: "이라고 안내문이 출력된 후, 프로그램 사용자가 "123456<엔터 키>"를 치면, 총 7바이트(마지막 바이트는 <엔터 키>에 해당하는 개행 문자의 아스키 코드 값)이 입력 버퍼로 저장되고, fgets는 최대 8-1=7바이트를 읽어오므로, "123456<엔터 키>"의 7바이트를 읽어온 후 마지막에 널 문자에 해당하는 0값의 한 바이트를 저장하여 총 8바이트의 값을 2차원 배열 id의 첫 행에 저장한다. 이 때 입력 버퍼에는 아무 것도 남아 있지 않은 상태가 된다.

두 번째 라인에서 "주민번호 뒤 7자리 입력: "이라고 안내문이 출력된 후, 프로그램 사용자가 "12345678<엔터 키>"를 치면, 총 9바이트(마지막 바이트는 <엔터 키>에 해당하는 개행 문자의 아스키 코드 값)이 입력 버퍼로 저장되고, fgets는 최대 8-1=7바이트를 읽어오므로, "1234567"의 7바이트를 읽어온 후 마지막에 널 문자에 해당하는 0값의 한 바이트를 저장하여 총 8바이트의 값을 2차원 배열 id의 두 번째 행에 저장한다. 이 때 입력 버퍼에는 아직 8과 <엔터 키>에 해당하는 2바이트의 데이터가 남아 있음에 주의해야 한다.

이 후 **printf("주민 번호: %s-%s\n", id[0], id[1]);** 문에 의해, 세 번째 라인과 네 번째 라인이 출력된다. id 배열의 첫 행에 저장된 문자열은 "123456<엔터 키>"이고, id 배열의 두 번째 행에 저장된 문자열은 "1234567"이므로, 세 번째 라인은 "주민 번호: 123456"이 되고 그 다음 줄 즉 네 번째 줄에 '-'와 id 배열의 두 번째 행이 연이어 출력되어 "-1234567"이 출력된다.

이 후 while문에 의한 두 번째 루프 시행에서 "주민번호 앞 6자리 입력: "이라고 안내문이 출력된 후, fgets 함수가 호출되면 직전에 입력했던 8과 <엔터 키>가 아직 입력 버퍼에 남아 있기 때문에 이것만 읽어 와서 널 문자를 첨부하여 '8', '\n'과 '\0'에 해당하는 3바이트를 id 배열의 첫 행에 저장한다. 그리고 나서 이 프로그램은 "주민번호 뒤 7자리 입력: "이라고 안내문을 출력하고, 프로그램 사용자의 입력을 기다린다. 그런데 프로그램 사용자가 E와 <엔터 키>를 입력하였기 때문에, id 배열의 두 번째 행에는 'E', '\n'과 '\0'에 해당하는 3바이트가 저장된다.

이 후 **printf("주민 번호: %s-%s\n", id[0], id[1]);** 문에 의해, 여섯 번째 라인부터 여덟 번째 라인까지 출력된다. id 배열의 첫 행에 저장된 문자열은 "8<엔터 키>"이고, id 배열의 두 번째 행에 저장된 문자열은 "E<엔터 키>"이므로, 여섯 번째 라인은 "주민 번호: 8"이 되고 그 다음 줄 즉 일곱 번째 줄에 '-'와 id 배열의 두 번째 행이 연이어 출력되어 "-E"가 출력된 후 여덟 번째 줄로 바뀐다. **printf("주민 번호: %s-%s\n", id[0], id[1]);** 문의 포맷 문자열 마지막에 개행 문자가 있으므로 이것이 출력되어 아홉 번째 줄로 바뀜에 따라, 여덟 번째 줄은 빈 줄이 된다.

이 후 while문에 의한 세 번째 루프 시행에서 "주민번호 앞 6자리 입력: "이라고 안내문이 출력되었을 때, 프로그램 사용자가 E와 <엔터 키>를 입력하였기 때문에, 시행 예의 마지막 라인으로서 END를 출력하고 프로그램이 종료된다.

(2) 프로그램과 시행 예에서 가려진 내용에 대한 답을 찾아가는 논리

fgets 함수 호출 부분으로부터 2차원 char 배열 id를 선언해야 한다는 것으로 알 수 있다. 최소한의 메모리를 지역변수에 사용하려면 id의 행수(세로길이)는 2가 되어야 한다.

시행 예의 3번째 줄에 123456이 출력되고 4번째 줄로 바뀌었기 때문에 첫 번째 fgets가 최소한 123456<엔터 키><널문자>의 8바이트를 id의 첫 행에 읽어 들였음을 알 수 있다. 이는 시행 예의 1번째 줄에서 프로그램 사용자가 123456<엔터 키>를 입력하였음을 의미한다. 따라서 문제 1-3의 답은 `123456`이다.

시행 예의 4번째 줄로부터 두 번째 fgets가 1234567<널 문자>의 8바이트만 읽어 들였고, 시행 예의 5번째 줄에서 "주민번호 앞 6자리 입력: " 안내가 출력된 후, 프로그램 사용자가 입력할 시간도 주지 않고 바로 연이어 "주민번호 뒤 7자리 입력: "이라는 안내가 출력된 것으로 미루어 직전 fgets 호출 이후 입력 버퍼에 뭔가 남아 있었고, 이번 fgets는 이 남아 있는 내용을 읽어 들였음을 알 수 있다. 시행 예의 6번째 및 7번째 줄로부터 상기 남아 있는 내용이 8과 <엔터 키>임을 알 수 있다. 따라서 시행 예의 2번째 줄에서 프로그램 사용자가 입력했던 내용이 12345678<엔터 키>임을 알 수 있다. 즉 문제 1-4의 답은 `12345678`이다. 이로써 두 번째 fgets가 id 배열의 열수(가로길이) 제한으로 인해 1234567<널 문자>의 8바이트만 읽어 들였음을 알 수 있다. 따라서 id 배역의 열수는 8임을 알 수 있다. 즉 문제 1-1의 답은 `char id[2][8];`이다.

시행 예의 9번째 줄로부터 프로그램 사용자가 "주민번호 앞 6자리 입력: "라는 안내문에 E<엔터 키>를 입력한 후, 프로그램이 10번째 줄에 END를 출력하고 프로그램을 종료하였으므로, 문제 1-2의 답은 `if (id[0][0] == 'E') break;`이다.

<u>문제 1 풀이 끝.</u>

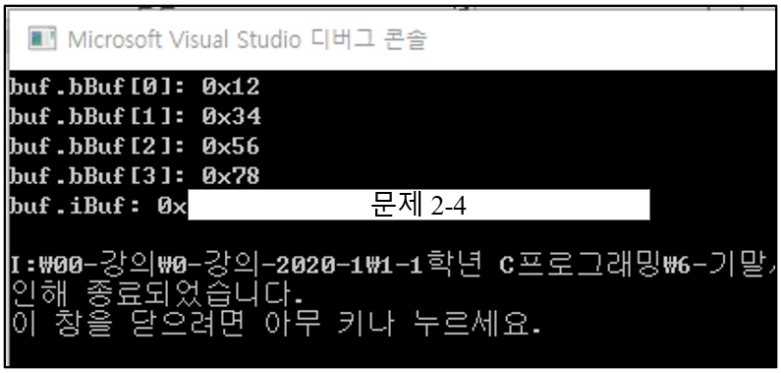
문제 2. (20점) 아래 소스 프로그램과 그 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

```
#include <stdio.h>
#include <string.h>

[redacted]
{
    unsigned char bBuf[4];
    unsigned short sBuf[2];
    unsigned int iBuf;
} RDBuf;

void main(void)
{
    RDBuf buf = { 0 };
    int i;

    buf.sBuf[0] = 0x[redacted];
    buf.sBuf[1] = 0x[redacted];

    for (i = 0; i < sizeof(buf.bBuf) / sizeof(unsigned char); i++)
        printf("buf.bBuf[%d]: %#x \n", i, buf.bBuf[i]);

    printf("buf.iBuf: %#x \n", buf.iBuf);
}
```

문제 2-1 (C 언어 키워드들임)

문제 2-2

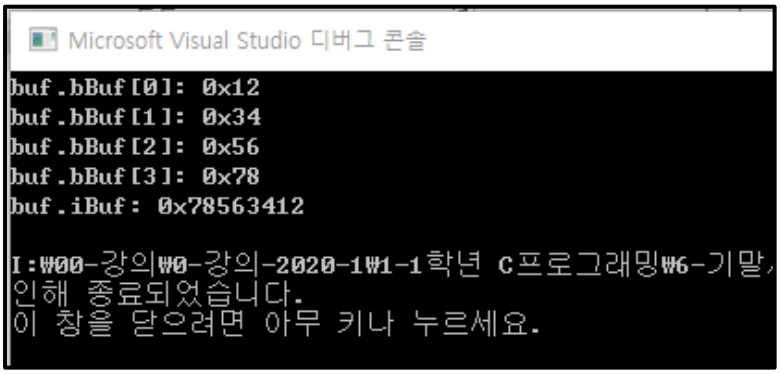
문제 2-3



문제 2 끝.

답:

```c
#include <stdio.h>
#include <string.h>

typedef union
{
    unsigned char bBuf[4];
    unsigned short sBuf[2];
    unsigned int iBuf;
} RDBuf;

void main(void)
{
    RDBuf buf = { 0 };
    int i;

    buf.sBuf[0] = 0x3412;
    buf.sBuf[1] = 0x7856;

    for (i = 0; i < sizeof(buf.bBuf) / sizeof(unsigned char); i++)
        printf("buf.bBuf[%d]: %#x \n", i, buf.bBuf[i]);

    printf("buf.iBuf: %#x \n", buf.iBuf);
}
```



문제 2-1의 답: **typedef union**

문제 2-2의 답: **3412**

문제 2-3의 답: **7856**

문제 2-4의 답: 78563412

채점 기준:

각 소문항 당 5점 배점. 문제 2-1의 경우, typedef를 누락하면 3점만 부여. 문제 2-2부터 2-4의 답에서 앞에 0x를 붙인 답도 감점하지 않음. 다른 부분 점수 없음

<u>풀이</u>:

(1) 일단 프로그램과 시행 예에서 가려진 내용을 모두 알고 있는 상태에서 프로그램의 동작을 설명한 후, 아래 (2)에서 답을 찾아가는 논리에 대해 설명하고자 한다.

인텔 CPU를 사용하는 시스템은 little-endian 방식으로 데이터를 메모리에 저장한다. 즉 4바이트 unsigned int형 변수를 저장한다고 할 때, LSB(Least-Significant Byte, 최하위 바이트)부터 MSB(Most-Significant Byte, 최상위 바이트)에 이르는 4바이트를 순차적으로 메모리에 저장한다. 즉 LSB의 주소지가 가장 작고, MSB의 주소지가 가장 크다. 또 2바이트 unsigned short형 변수를 저장한다고 할 때에도 LSB가 먼저 저장되고 그 다음 주소지에 MSB가 저장된다.

공용체의 멤버들은 메모리 공간을 공유한다.

2바이트의 unsigned short형 배열인 sBuf 멤버의 첫 요소에 0x3412를, 두 번째 요소에 0x7856을 저장하였으므로, RDbuf 공용체 변수 buf가 차지하는 4바이트의 메모리 공간에는 작은 주소지부터 0x12, 0x34, 0x56, 0x78이 차례로 저장된다. 따라서 이를 1바이트 unsigned char형 배열인 bBuf 멤버를 통해 차례로 읽어 내면, 시행 예의 1번째부터 4번째 줄의 내용과 같다.

같은 메모리 영역을 4바이트 unsigned int형 변수인 iBuf 멤버를 통해 접근하면, 0x78이 MSB가 되고 0x12가 LSB가 되어, 0x78563412로 읽힌다.

(2) 프로그램과 시행 예에서 가려진 내용에 대한 답을 찾아가는 논리

buf.bBuf 등의 **.** 연산자를 사용하는 표현이 등장하므로 **buf** 변수의 타입은 struct가 아니면 union 중 한 가지이다. 프로그램 초기에 **RDBuf buf = { 0 };**를 실행하여 **buf** 변수의 모든 멤버를 0으로 초기화한 후, **buf.bBuf** 멤버 배열의 값을 수정한 적이 없음에도 불구하고, 시행 예의 1번째부터 4번째 출력 줄로부터 이 값들이 0이 아님을 관찰할 수 있다. 따라서 **buf.bBuf** 멤버 배열은 **buf**의 다른 멤버와 메모리를 공유하고 있음을 알 수 있다. 따라서 **buf**는 공용체 변수이다. 그러므로 **RDBuf**는 공용체에 해당하는 사용자 정의 형이 되어야 한다. 즉 문제 2-1의 답은 **typedef union**이다.

시행 예의 1번째부터 4번째 줄의 내용이 **buf.bBuf** 배열의 값을 알려 주므로, 상기 (1)의 논의로부터, 문제 2-2와 문제 2-3의 답은 각기 **3412**와 **7856**이다. 또한 문제 2-4의 답은 78563412이다.

<u>**문제 2 풀이 끝.**</u>

문제 3. (20점) 아래 소스 프로그램과 그 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

```
#include <stdio.h>

void readWrite(const char *file,
     const char *writeMode, const char *readMode,
     const void *data, int dataLen)
{
     FILE *fp;
     unsigned char c;

     fp = fopen(file, writeMode);
     fwrite(data, 1, dataLen, fp);
     fclose(fp);

     fp = fopen(file, readMode);
     while (fread(&c, 1, 1, fp) == 1)
         printf("%02X ", c);
     printf("\n");
     fclose(fp);
}

void main(void)
{
     unsigned char arr[ ] = [redacted];

     readWrite("testfile", "wb", "rb", arr, sizeof(arr));
     readWrite("testfile", "wt", "rb", arr, sizeof(arr));
     readWrite("testfile", "wb", "rt", arr, sizeof(arr));
     readWrite("testfile", "wt", "rt", arr, sizeof(arr));
}
```

문제 3-1 (숫자가 포함되어 있다면, 반드시 십진수로 답해야 함)

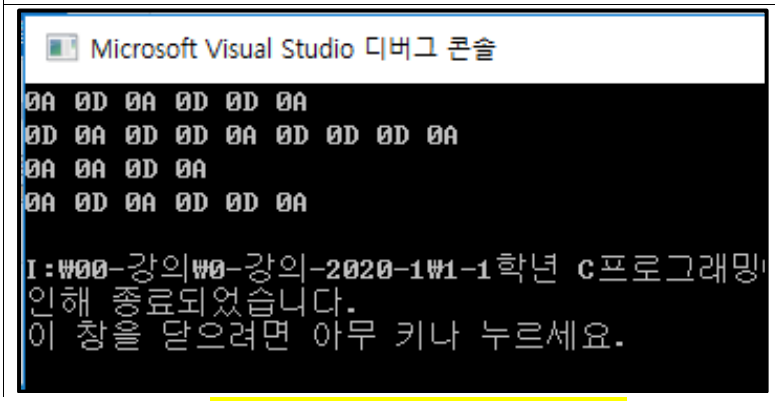
Microsoft Visual Studio 디버그 콘솔

```
문제 3-2
0D 0A 0D 0D 0A 0D 0D 0D 0A
문제 3-3
문제 3-4

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로그래밍
인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 3 끝.

답:

```c
#include <stdio.h>

void readWrite(const char *file,
    const char *writeMode, const char *readMode,
    const void *data, int dataLen)
{
    FILE *fp;
    unsigned char c;

    fp = fopen(file, writeMode);
    fwrite(data, 1, dataLen, fp);
    fclose(fp);

    fp = fopen(file, readMode);
    while (fread(&c, 1, 1, fp) == 1)
        printf("%02X ", c);
    printf("\n");
    fclose(fp);
}

void main(void)
{
    unsigned char arr[ ] = { 10, 13, 10, 13, 13, 10 };

    readWrite("testfile", "wb", "rb", arr, sizeof(arr));
    readWrite("testfile", "wt", "rb", arr, sizeof(arr));
    readWrite("testfile", "wb", "rt", arr, sizeof(arr));
    readWrite("testfile", "wt", "rt", arr, sizeof(arr));
}
```



문제 3-1의 답: `{ 10, 13, 10, 13, 13, 10 }`

문제 3-2의 답: `0A 0D 0A 0D 0D 0A`

문제 3-3의 답: `0A 0A 0D 0A`

문제 3-4의 답: `0A 0D 0A 0D 0D 0A`

채점 기준:

각 소문항 당 5점 배점. 문제 3-1에서 중괄호가 누락되었으면, 4점만 부여(즉, 1점 감점)함. 문제 3-2부터 문제 3-3의 경우, "0D"를 "0d"라고 쓰거나, "0A"를 "0a"로 쓴 답은 1점 감점. 또 "0D"를 "D"라고 쓰거나, "0A"를 "A"로 쓴 답도 1점 감점. 만약 "0D"를 "d"라고 쓰거나, "0A"를 "a"로 쓴 답은 2점 감점. 이러한 감점은 문제 3-2부터 문제 3-4에 걸쳐 여러 번 나오더라도 문제 3-2부터 문제 3-4까지 전체에 대해 1회만 적용함. 예를 들어 문제 3-2와 문제 3-3의 답만 맞았는데, 상기 1점 감점할 사항이 문제 3-2에도 또 문제 3-3에도 포함되어 있다면, 5점x2문항=10점으로부터 1점을 감점하여 9점을 부여함. 다른 부분 점수 없음

풀이:

(1) 일단 프로그램과 시행 예에서 가려진 내용을 모두 알고 있는 상태에서 프로그램의 동작을 설명

한 후, 아래 (2)에서 답을 찾아가는 논리에 대해 설명하고자 한다.

텍스트 모드와 바이너리 모드의 차이는 파일로 데이터를 써넣거나 읽어 올 때, 조작 여부에 달려 있다. OS에 따라 텍스트 모드에서 개행 문자 '\n'(0x0A)에 한하여 이러한 조작을 시행하지만, 바이너리 모드의 경우 어떤 OS에서도 조작을 하지 않는다. 윈도우즈 OS의 경우, '\n'을 파일로 써넣을 때, '\r'(0x0D)과 '\n'(0x0A)의 2바이트를 순차적으로 저장하게 된다. 또 파일로부터 읽어 올 때에는 0x0D0A의 2바이트를 1바이트의 개행 문자 즉 '\n'(0x0A)으로 읽어 온다.

텍스트 모드와 바이너리 모드를 결정하는 것은 파일을 개방할 때의 모드 문자열 인자이며, fread, fwrite, fgetc, fputc 등의 표준함수가 이를 결정하는 것은 아니다.

파일을 텍스트 모드로 쓰기 위해 개방한 경우, 개행 문자 '\n'(0x0A)에 해당하는 한 바이트 값을 저장하려고 하면 윈도우 OS에서는 '\r'(0x0D)과 '\n'(0x0A)을 순차적으로 저장하게 된다. 따라서 십진수로 10, 13, 10, 13, 13, 10(16진수로는 0A, 0D, 0A, 0D, 0D, 0A)을 차례대로 저장하면, 실제로 파일에는 0D, 0A, 0D, 0D, 0A, 0D, 0D, 0D, 0A가 차례로 저장된다. 이렇게 저장된 파일 내용을 바이너리 모드로 읽기위해 파일 개방하여 이진 데이터로서 읽어서 모니터로 출력하면, 파일에 들어 있는 바이트 데이터들이 모두 출력된다. 즉 이 경우, 0D 0A 0D 0D 0A 0D 0D 0D 0A가 출력된다.

파일을 바이너리 모드로 쓰기 위해 개방한 경우, 모든 바이트 값은 어떤 수정도 없이 1바이트로 저장된다. 이후 이 파일을 텍스트 모드로 개방하여 읽을 때, 파일에 들어 있는 연속된 두 바이트 값이 0x0D('\r'), 0x0A('\n')이면, 0x0A('\n')의 한 바이트 즉 개행 문자로 변환되어 읽힌다. 따라서 파일 내에 십진수로 10, 13, 10, 13, 13, 10(16진수로는 0A, 0D, 0A, 0D, 0D, 0A)이 차례대로 저장되어 있는 경우, 이를 텍스트 모드로 차례로 읽으면 첫 번째 바이트 0A는 그냥 '\n' 문자로 읽히고, 두 번째 바이트와 세 번째 바이트에 해당하는 0D와 0A는 하나의 '\n' 문자로 읽히고, 세 번째 바이트 0D는 그냥 '\r' 문자로 읽히며, 다섯 번째와 여섯 번째 바이트인 0D, 0A는 하나의 '\n' 문자로 읽힌다. 따라서, '\n', '\n', '\r', '\n'의 4개 문자가 순차적으로 읽힌다. 이를 16진수로 출력하면 0A가 2번 출력된 후 이어서 0D와 0A가 각기 1번씩 출력된다. 즉 이 경우 0A 0A 0D 0A가 출력된다.

파일을 바이너리 쓰기 모드로 개방하여 데이터를 써넣고, 다시 이를 바이너리 읽기 모드로 개방하여 읽으면 써넣었던 데이터가 그대로 읽힌다.

또한 파일을 텍스 쓰기 모드로 개방하여 데이터를 써넣고, 다시 이를 텍스트 읽기 모드로 개방하여 읽은 경우에도 써넣었던 데이터가 그대로 읽힌다. 왜냐하면, 0A를 0D 0A로 써넣었다가 읽어 올 때는 0D 0A를 0A로 바꿔서 읽어 오기 때문이다.

(2) 프로그램과 시행 예에서 가려진 내용에 대한 답을 찾아가는 논리

시행 예의 2번째 줄은 텍스트 모드로 써넣고 바이너리 모드로 읽어서 출력한 결과이므로, 이 출력 내용을 0D, 0A, 0D, 0D, 0A, 0D, 0D, 0D, 0A와 같이 밑줄 부분을 기준으로 변환하여 보면, 원래 써넣은 데이터가 10, 13, 10, 13, 13, 10임을 알 수 있다. 따라서 문제 3-1의 답은 **{ 10, 13, 10, 13, 13, 10 }**이다.

시행 예의 1번째 줄은 바이너리 모드로 써넣고 바이너리 모드로 읽어서 출력한 결과이므로, 원래 데이터가 그대로 출력된다. 따라서 10, 13, 10, 13, 13, 10이 16진수로 출력된다. 따라서 문제 3-2의 답은 0A 0D 0A 0D 0D 0A이다.

시행 예의 3번째 줄은 바이너리 모드로 써넣고 텍스트 모드로 읽어서 출력한 결과이므로, 원래

데이터 10, 13, 10, 13, 13, 10이 파일에 그대로 써진 결과인 0A, 0D, 0A, 0D, 0D, 0A로부터 밑줄을 기준으로 변환하여 읽어 오면 0A, 0A, 0D, 0A가 출력된다. 따라서 문제 3-3의 답은 0A 0A 0D 0A이다.

시행 예의 4번째 줄은 텍스트 모드로 써넣고 텍스트 모드로 읽어서 출력한 결과이므로, 원래 데이터가 그대로 출력된다. 따라서 10, 13, 10, 13, 13, 10이 16진수로 출력된다. 따라서 문제 3-4의 답은 문제 3-2의 답과 동일하게 0A 0D 0A 0D 0D 0A이다.

문제 3 풀이 끝.

문제 4. (20 점) 아래 소스 프로그램과 시행 예를 보고, 소스 프로그램에서 가려진 부분의 내용을 문제 번호와 함께 써라.

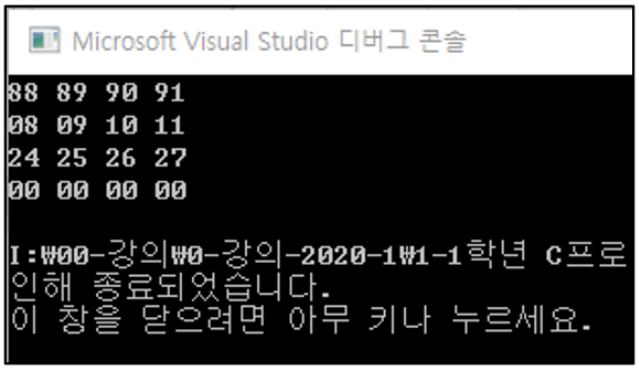
```
#include <stdio.h>

void setArray(int *arr, int count, int init)
{
    int i;

    for (i = 0; i < count; i++)
        arr[i] = (init++) % [redacted];
}

void printArray(const int(*arr)[[redacted]], int y)
{
    int i, j;

    for (i = 0; i < y; i++)
    {
        for (j = 0; j < 4; j++)
            printf("%02d ", arr[i][j]);
        printf("\n");
    }
}

void main(void)
{
    int arr[12][[redacted]] = { 0 };

    setArray((int *)arr, (sizeof(arr[0]) / sizeof(int)) *
        (sizeof(arr) / sizeof(arr[0]) - 1), [redacted]);

    printArray((void *)&arr[5], 4);
}
```

문제 4-1 (정수이므로, 반드시 십진수로 답해야 함)

문제 4-2 (정수이므로, 반드시 십진수로 답해야 함)

문제 4-3 (정수이므로, 반드시 십진수로 답해야 함)

문제 4-4 (정수이므로, 반드시 십진수로 답해야 함)



문제 4 끝.

답:

```c
#include <stdio.h>

void setArray(int *arr, int count, int init)
{
    int i;

    for (i = 0; i < count; i++)
        arr[i] = (init++) % 96;
}

void printArray(const int(*arr)[16], int y)
{
    int i, j;

    for (i = 0; i < y; i++)
    {
        for (j = 0; j < 4; j++)
            printf("%02d ", arr[i][j]);
        printf("\n");
    }
}

void main(void)
{
    int arr[12][8] = { 0 };

    setArray((int *)arr, (sizeof(arr[0]) / sizeof(int)) *
        (sizeof(arr) / sizeof(arr[0]) - 1), 48);

    printArray((void *)&arr[5], 4);
}
```

문제 4-1의 답: **96**

문제 4-2의 답: **16**

문제 4-3의 답: **8**

문제 4-4의 답: **48**

채점 기준:

각 소문항 당 5점 배점. 부분 점수 없음

풀이:

(1) 일단 프로그램과 시행 예에서 가려진 내용을 모두 알고 있는 상태에서 프로그램의 동작을 설명한 후, 아래 (2)에서 답을 찾아가는 논리에 대해 설명하고자 한다.

이 문제는 2차원 배열의 요소들이 메모리 상에서 어떻게 저장되는지를 확인하기 위한 것이다. 2차원 배열의 첫 행부터 마지막 행까지 메모리에 순차적으로 저장되므로, 2차원 배열의 시작 주소값을 1차원 배열의 시작 주소값이라고 해석하여 1차원 배열인 것처럼 처리할 수 있다. 사실 이렇게 처리하는 것이 더 편리할 때가 많은데, 문제의 경우와 같이, 어떤 함수로 2차원 배열의 주소값을 넘겨주려면, 데이터 타입을 지정할 때 열의 개수를 명시적으로 지정해야 한다. (상기 문제에서 **void printArray(int (*arr)[16], int y)** 부분에 주목) 필요에 따라 프로그램을 수정할 때, 2차원 배열의 크기를 수정하게 되면, 함수 정의 부분도 수정해야 하므로 프로그램 관리가 불편할 수 있다.

이 문제에서는 총 12*8=96개 요소를 갖는 2차원 배열의 모든 요소를 0으로 초기화한 후, 1차원 배열인 것으로 간주하여 (48 modulo 96)=48로부터 ((48+((12-1)*8-1)=135) modulo 96)=39까지의 값

을 11번째 행까지(마지막 행 제외)의 각 요소에 저장한 후, 같은 메모리 공간을 6번째 행부터 시작하여 열수(가로길이)가 16인 2차원 배열로 접근하되 4개 행의 첫 4개 요소(이는 **main** 함수의 **arr** 배열을 기준으로 보면, 6번째 행부터 11번째 행까지의 8개 행 중에서 짝수 번째 행의 첫 4요소에 해당함)를 행 별로 별도 줄에 출력하게 하였다. 이 때, **printArray** 함수의 첫 파라미터인 배열 포인터의 열수는 16으로서, 이를 호출한 메인 함수의 인자(열수가 8인 배열 주소)와 형이 다르다. 이로 인해, **printArray** 함수에서는 넘겨 받은 주소 값을 열수가 16인 배열의 주소 값으로 해석한다.

넘겨 받은 주소 값은 메인 함수에서 선언된 2차원 배열 **int arr[12][8]**의 메모리 공간에서 41번째에 해당하는 정수 주소 값이므로, 해당 주소지에는 ((48+40=88) modulo 96)=88이 저장되어 있다. 따라서 1번째 줄에 출력되는 4개의 값은 ((48+40=88) modulo 96)=88부터 ((48+(40+3)=88) modulo 96)=91까지이다. 그러므로 1번째 줄에는 `88 89 90 91`이 출력된다. (이는 **main** 함수의 **arr** 배열을 기준으로 보면, 6번째 행의 첫 4요소에 해당함)

2번째 줄에 출력되는 4개의 값은 ((48+40+16=88) modulo 96)=8부터 ((48+(40+16+3)=88) modulo 96)=11까지이다. 그러므로 2번째 줄에는 `08 09 10 11`이 출력된다. (이는 **main** 함수의 **arr** 배열을 기준으로 보면, 8번째 행의 첫 4요소에 해당함)

3번째 줄에 출력되는 4개의 값은 ((48+40+16*2=88) modulo 96)=24부터 ((48+(40+16*2+3)=88) modulo 96)=27까지이다. 그러므로 3번째 줄에는 `24 25 26 27`이 출력된다. (이는 **main** 함수의 **arr** 배열을 기준으로 보면, 10번째 행의 첫 4요소에 해당함)

4번째 줄에 출력되는 4개의 값은 **main** 함수의 **arr** 배열을 기준으로 보면, 12번째 행의 첫 4요소에 해당하므로 모두 0이다. 그러므로 4번째 줄에는 `00 00 00 00`이 출력된다.

(2) 프로그램과 시행 예에서 가려진 내용에 대한 답을 찾아가는 논리

문제 4-1부터 4-4까지의 답을 각기 w, x, y, z라고 하자.

시행 예의 1번째 줄의 첫 출력 값으로부터 식을 수립하면, ((z+5y) modulo w)=88이다. 이를 식1이라 하자.

시행 예의 2번째 줄의 첫 출력 값으로부터 식을 수립하면, ((z+5y+x) modulo w)=8이다. 이를 식2라 하자.

시행 예의 3번째 줄의 첫 출력 값으로부터 식을 수립하면, ((z+5y+2x) modulo w)=24이다. 이를 식3이라 하자.

식2와 식3으로부터, x=24-8=16이다. 따라서 문제 4-2의 답은 **16**이다.

식1과 식2로부터 (88+16) modulo w = 8이므로, w=104-8=96이다. 따라서 문제 4-1의 답은 **96**이다.

시행 예의 4번째 줄의 출력 값이 모두 0이므로 이는 **main** 함수의 **arr** 배열을 기준으로 보면, 마지막 행의 값들에 해당한다. (만약 출력된 값이 **main** 함수의 **arr** 배열의 메모리 범위를 넘어선 곳의 값이었다면 쓰레기 값이 출력되었을 것이다.) 따라서 5y + (4-1)*16 >= (12-1)y이고 5y + (4-1)*16 + (3-1) >= 12y-1이어야 한다. 따라서 52/7 <= y <= 8이 되는데, 이를 만족하는 정수 y는 8뿐이다. 따라서 y=8 즉 문제 4-3의 답은 **8**이다.

식1로부터 ((z+5*8) modulo 96)=88이므로 z=88-40=48이다. 따라서 문제 4-4의 답은 **48**이다.

문제 4 풀이 끝.

문제 5. (20 점) 아래 소스 프로그램과 그 실행 예에서 가려진 부분의 내용을 문제 번호와 함께 써라.

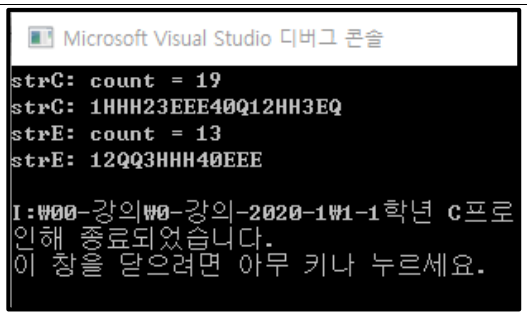
```
#include <stdio.h>
#include <string.h>

#define STRING1 "It's very strange??"   // Every blank is "a single space."
#define STRING2 " --- I'm confused!!!" // Every blank is "a single space."
#define STRING3 \
    "Is anybody there? Um -- Nobody!?" // Every blank is "a single space."

int count;

void bbb(char **ptr)
{
    static unsigned char num = 0;

    if (!ptr)
    {
        num = 0;
        return;
    }
    num++; num %= 5;
    **ptr = '0' + num; (*ptr)++; count++;
}

void eee(char **ptr) { **ptr = 'E'; (*ptr)++; count++; }
void hhh(char **ptr) {   **ptr = 'H'; (*ptr)++; count++; }
void qqq(char **ptr) {   **ptr = 'Q'; (*ptr)++; count++; }
void nnn(char **ptr) {   **ptr = '\0'; }

void main(void)
{
    char *strA = STRING1;
    char *strB = STRING2;
    char strC[] = STRING2 STRING3;
    char *strD = STRING3;
    char strE[] = STRING1 STRING2;

    char *ptrS[] = { strA, strB, strC, strD, strE }, *p, *pp;
    [redacted] = { bbb, eee, hhh, qqq, nnn };
    int i;
                                    문제 5-1

    for (i = 0; i < sizeof(ptrS) / sizeof(ptrS[0]); i++) {
        p = pp = ptrS[i];
        if (p == STRING1 || p == STRING2 || p == STRING3)
            continue;
        for ([redacted]; ; p++) {
                                                    문제 5-2
            switch (*p)
            {
            case ' ':    func[0](&pp); break;
            case '!':    func[1](&pp); break;
            case '-': func[2](&pp); break;
            case '?':    func[3](&pp); break;
            case '\0': func[4](&pp); break;
            }
            if (*p == '\0') break;
        }
        printf("str%c: count = %d\n", 'A' + i, count);
        printf("str%c: %s\n", 'A' + i, *(ptrS + i));
    }
}
```

```
Microsoft Visual Studio 디버그 콘솔
            문제 5-3
            문제 5-4
strE: count = 13
strE: 12QQ3HHH40EEE

I:₩00-강의₩0-강의-2020-1₩1-1학년 C프로
인해 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요.
```

문제 5 끝.

답:

```c
#include <stdio.h>
#include <string.h>

#define STRING1 "It's very strange??"   // Every blank is "a single space."
#define STRING2 " --- I'm confused!!!" // Every blank is "a single space."
#define STRING3 \
    "Is anybody there? Um -- Nobody!?" // Every blank is "a single space."

int count;

void bbb(char **ptr)
{
    static unsigned char num = 0;

    if (!ptr)
    {
        num = 0;
        return;
    }
    num++; num %= 5;
    **ptr = '0' + num; (*ptr)++; count++;
}

void eee(char **ptr) { **ptr = 'E'; (*ptr)++; count++; }
void hhh(char **ptr) {   **ptr = 'H'; (*ptr)++; count++; }
void qqq(char **ptr) {   **ptr = 'Q'; (*ptr)++; count++; }
void nnn(char **ptr) {   **ptr = '\0'; }

void main(void)
{
    char *strA = STRING1;
    char *strB = STRING2;
    char strC[] = STRING2 STRING3;
    char *strD = STRING3;
    char strE[] = STRING1 STRING2;

    char *ptrS[] = { strA, strB, strC, strD, strE }, *p, *pp;
    void (*func[])(char **) = { bbb, eee, hhh, qqq, nnn };
    int i;

    for (i = 0; i < sizeof(ptrS) / sizeof(ptrS[0]); i++) {
        p = pp = ptrS[i];
        if (p == STRING1 || p == STRING2 || p == STRING3)
            continue;
        for (func[0](NULL), count = 0; ; p++) {
            switch (*p)
            {
            case ' ':    func[0](&pp); break;
            case '!':    func[1](&pp); break;
            case '-': func[2](&pp); break;
            case '?':    func[3](&pp); break;
            case '\0': func[4](&pp); break;
            }
            if (*p == '\0') break;
        }
        printf("str%c: count = %d\n", 'A' + i, count);
        printf("str%c: %s\n", 'A' + i, *(ptrS + i));
    }
}
```



문제 5-1의 답: `void (*func[])(char **)` 또는 `void (*func[5])(char **)`
문제 5-2의 답: `func[0](NULL), count = 0` 또는 `count = 0, func[0](NULL)`
문제 5-3의 답: `strC: count = 19`
문제 5-4의 답: `strC: 1HHH23EEE40Q12HH3EQ`

<u>채점 기준</u>:

각 소문항 당 5점 배점. 문제 5-2의 답에서 **NULL** 대신 **(char **)NULL** 또는 **0**이라고 기재한 답도 맞는 답으로 처리하되, **func[0](NULL)**과 **count = 0** 중 하나만 답하면 3점만 부여함. 문제 5-3의 답에서 "strC:" 및/또는 "count ="를 누락하면 1점 감점함. 단, 빈 칸 표시에 사소한 오류가 있어 보이는 경우는 맞는 답으로 처리함. 5-4의 답에서 "strC:"를 누락하면 1점 감점함. 다른 부분 점수 없음

<u>풀이</u>:

(1) 일단 프로그램과 시행 예에서 가려진 내용을 모두 알고 있는 상태에서 프로그램의 동작을 설명한 후, 아래 (2)에서 답을 찾아가는 논리에 대해 설명하고자 한다.

문제의 프로그램은 주어진 문자열에서 ' '(빈 칸)이 등장할 때마다 '1'부터 '4' 또는 '0' 중 하나를, '!'(Exclamation), '-'(Hyphen), 그리고 '?'(Question mark) 중 하나가 등장할 때마다 각기 'E', 'H', 'Q'를, 해당 문자열의 처음부터 기재(원래 들어 있던 내용을 덮어씌움)하고 마지막에 널 문자를 기재한다. ' '이 등장할 때, 기재되는 수는 각 문자열에 대해, 처음 ' '이 등장하면 '1', 그 다음 등장 시 '2' 등으로 증가하여 '4' 다음에는 '0', 그리고 다시 '1'부터 시작하도록 되어 있다. 또한 기재하는 문자의 수는 널 문자를 제외하고 센다. 다섯 개의 문자열에 대해 각기 이러한 작업을 수행 후, 기재된 문자 수와 변경된 문자열을 출력하는데, 1번째, 2번째 그리고 4번째 문자열은 문자열 상수로서 그 메모리 공간이 이 프로그램이 변경해서는 안 되는 영역에 있기 때문에, 이러한 점을 검사하여, 만약 그렇다면, 이 프로그램이 변경해서는 안 되는 영역에 '덮어씌움'을 하면 안 되므로 이에 대해서는 아무런 작업도 하지 않으며 물론 아무 것도 출력하지 않는다. 이 때, 상기 '검사' 방법은 주어진 문자열의 주소가 어떤 문자열 상수의 주소와 동일한지 검사하는 것이다. 만약 참이면, 문자열 상수에 해당하므로 아무런 처리를 하지 않고 다음 문자열을 처리하기 위해 넘어 간다.

참고로, 상기 프로그램의 메인 함수에서 **char *strA = STRING1;**이라는 문장은 문자열 상수 **STRING1**의 주소를 포인터 **strA**에 저장하는 문장이다. 따라서 포인터 **strA**에 저장된 값은 **STRING1**의 주소 값과 동일하다. 마찬가지로 **strB** 및 **strD**에 저장된 값은 각기 **STRING2** 및 **STRING3**의 주소 값과 동일하다.

반면 **char strC[] = STRING2 STRING3;** (**STRING2**와 **STRING3**의 두 매크로는 각기 문자열이므로 이를 연속으로 기재하면 해당되는 문자열 두 개를 이어 붙여 만든 하나의 문자열과 동일한 것으로 컴파일되며, 그 결과는 하나의 문자열 상수로 간주된다.)이라는 문장은 문자열 배열 **strC**의 내용을 **STRING2**와 **STRING3**이 연접된 문자열로 초기화하는 문장이다. 따라서 이 문장을 수행하고 나면, 이 프로그램의 메모리 영역 중 스택 영역으로부터 **STRING2**와 **STRING3**이 연접된 문자열을 저장할 만큼의 메모리가 **strC** 배열에 할당되고 **STRING2**와 **STRING3**이 연접된 문자열의 내용이 **strC** 배열로 복사된다. 따라서 **STRING2**와 **STRING3**이 연접된 문자열의 주소 값은 **strC**와 서로 다르다. 마찬가지로 **strE**와 **STRING1 STRING2**의 주소 값은 서로 다르다.

또 상기 '검사'를 수행하는 프로그램 코드 중, **p == STRING1**이란 표현이 나오는데, 이에 대해 수업시간 중에 강조하지는 않았으나, **p**가 포인터이므로 이를 **STRING1**과 비교한다는 것은 포인터 **p**에 저장된 주소 값을 **STRING1**의 주소지와 비교한다는 의미이다. 실제로 상기 프로그램의 **if (p == STRING1 || p == STRING2 || p == STRING3)** 문장을 **if (p == &STRING1 || p == &STRING2 || p == &STRING3)**이라는 문장으로 수정하여도 같은 결과를 얻는다.

참고로 이 프로그램에서는 함수 포인터 배열(function pointer array) func이 사용되었다.

(2) 프로그램과 시행 예에서 가려진 내용에 대한 답을 찾아가는 논리

문제 5-1에 해당하는 C 문장에서 **bbb, eee, hhh, qqq, nnn** 등은 모두 함수의 이름인데 이들을 초기화하는 값으로서 사용하였기 때문에, 선언하는 대상은 함수 포인터 배열이라야 한다. 이 문장 아래 부분에서 **func**이 여러 번 사용되었지만 이를 선언하는 문장이 없으므로 문제 5-1이 포함된 C 문장은 이를 선언하는 문장이 되어야 한다. 따라서 **bbb, eee, hhh, qqq, nnn** 등의 함수의 인자 수와 그 형, 그리고 반환값 형을 고려하여 **func**이라는 이름의 함수 포인터 배열을 선언하면 문제 5-1의 답을 얻을 수 있는데 이는 **void (*func[])(char **)**이다.

시행 예를 3번째 출력 줄로부터 count 값은 시행 예의 4번째 줄에 출력된 글자 수(즉, 배열에 기재된 글자 수)임을 알 수 있다. 따라서 count 값은 처리 대상 문자열마다 초기화되어야 함을 알 수 있다. 따라서 문제 5-2의 답에는 **count = 0**가 포함되어야 한다.

count는 전역 변수이므로 초기값이 0이며, 문제 5-2의 답에서 **count**를 0으로 또 초기화하므로, 문제 5-3의 답에 해당하는 **count** 값을 처음으로 증가시키기 직전 값으로서 0 이외의 값을 상정하는 것을 부자연스럽다. (즉 문제 5-2에 해당되는 답 내에서 **count = 0**이라고 한 직후 이어서 또 **count**를 다른 값으로 설정하는 것은, 그렇게 할 수는 있지만, 그 자체가 부자연스럽다는 뜻임) 따라서 상기 (1)에서 설명한 바에 따라 문제 5-3과 문제 5-4의 답을 구하면 count = 19 및 1HHH23EEE40Q12HH3EQ가 된다.

그런데 문제 5-4의 답을 구하는 과정에서 처리 대상 문자열에 빈 칸이 포함되어 있을 때 기재되는 정수가 3까지 진행되었는데, 시행 예의 4번째 줄의 출력 내용을 보면 이 정수가 static 변수 **num**에 의해 결정됨에도 불구하고 그 다음 수인 4로 이어지지 않고 1부터 다시 시작하였음을 관찰할 수 있다. 이로부터 처리 대상 문자열마다 이 정수를 초기화하고 있음을 알 수 있다. 따라서 문제 5-2에는 **count = 0** 뿐만 아니라 이 초기화를 시행하는 프로그램 코드가 포함되어야 한다. 빈 칸 대신 문자열에 기재되는 정수를 초기화하려면, 인수의 값을 **NULL**로 설정하여 **func[0]**를 호출해야 한다. 따라서 최종적으로 문제 5-2의 답은 **func[0](NULL), count = 0**이다.

문제 5 풀이 끝.

풀이 전체 끝.